Software Architecture & Design Specification

Bank Management System

Team E11

Meghna Sanjeev PES1UG24CS269

Mrinmayi Raman PES1UG24CS278

Advika Raj PES1UG24CS906

Manasi Vipin PES1UG24CS260

1. Introduction

This document defines the architecture and detailed design needed to implement the Bank Management System described in the SRS. The design uses a simple modular layered structure so the team can implement it without an external database, network service or REST API.

1.1 Design Goals

- Keep implementation simple enough for an academic mini-project.
- Separate user interaction, authentication, business rules and file storage.
- Make each SRS requirement traceable to a small set of modules.
- Provide enough structure for testing without introducing unnecessary frameworks.

2. Architecture

2.1 Architectural Pattern

The system follows a simple layered/modular architecture: Presentation Layer → Service/Business Layer → Persistence Layer. Authentication is a shared service used by employee and customer flows. This is intentionally not a distributed or web architecture.

2.2 Architecture Diagram

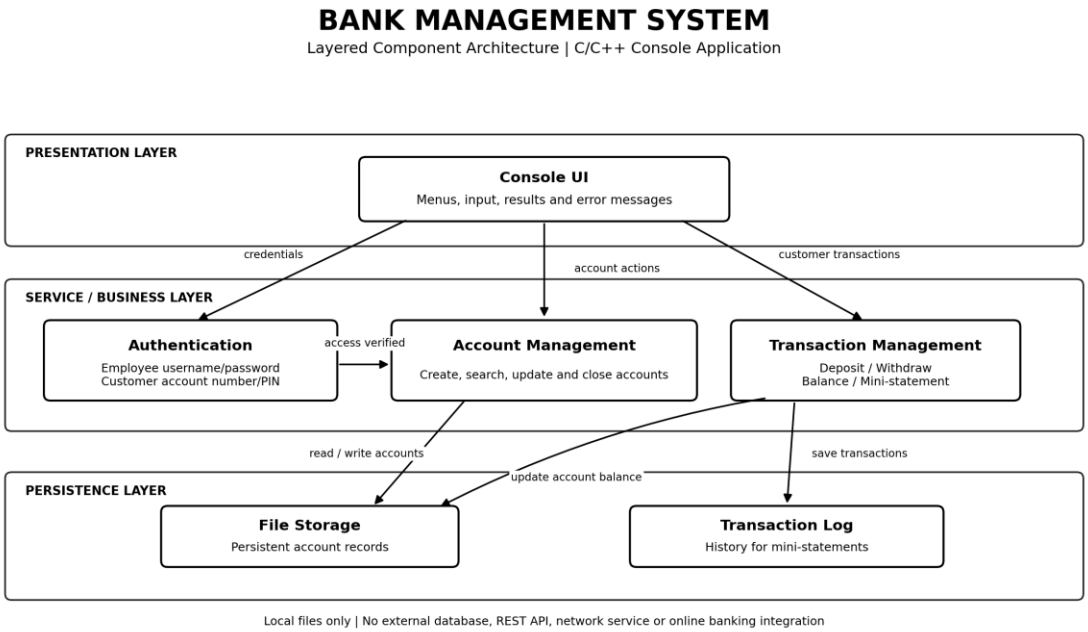


Figure 1. Architecture diagram.

2.3 Component Descriptions

Component	Responsibility
Console UI	Displays menus, reads input, validates basic format and displays results.
Authentication	Checks employee username/password and customer account number/PIN.
Account Management	Creates, searches, updates and closes accounts; generates account numbers.
Transaction Management	Validates deposits/withdrawals, updates balances and creates transaction entries.
File Storage	Reads and writes persistent account records.
Transaction Log	Stores transaction entries used by the mini-statement.

2.4 Data Flow

1. User selects an operation from the console.
2. Console UI collects input and sends it to the relevant module.
3. Authentication verifies access where required.
4. Business modules validate the operation and update the in-memory record.
5. File Storage persists the changed account/transaction data.
6. The result is returned to the console for display.

2.5 Architecture Traceability

Module	Main SRS Requirements
Console UI	FR-1 to FR-12, NFR-2
Authentication	FR-10, FR-11, SEC-1, SEC-2, SEC-3
Account Management	FR-1, FR-2, FR-7, FR-8, FR-9

Transaction Management	FR-3, FR-4, FR-5, FR-6, SEC-4
File Storage / Transaction Log	FR-13, NFR-3, NFR-6

3. Security Architecture

Security is implemented through simple access-control checks rather than a complex security framework.

Control	Design
Employee access	Employee login is checked before administrative menu operations.
Customer access	Account number + PIN is checked before self-service operations.
Failed authentication	The requested operation is not executed and an error is displayed.
Transaction validation	Only positive amounts are accepted; withdrawal balance is checked before update.
Scope limitation	The system is an academic local simulation and does not claim production banking security.

4. Detailed Design

4.1 Data Structures

Structure	Suggested Fields
Account	accountNumber, name, address, contactNumber, pin, balance, status
Transaction	transactionId, accountNumber, type, amount, date/time
Employee	username, password

These are suggested simple structures, not a commitment to a particular file format. The implementation team may use C structs or C++ classes.

4.2 API Design — Internal Module Interfaces

Because the project is a console application, 'API Design' refers to internal function/module interfaces rather than REST or web APIs.

Module	Suggested Function Interface	Purpose
Authentication	loginEmployee(); authenticateCustomer(accountNo, pin)	Return success/failure for access checks.
Account Management	createAccount(); searchAccount(accountNo); updateAccount(); closeAccount(accountNo)	Perform employee account operations.
Transactions	deposit(accountNo, amount); withdraw(accountNo, amount); getBalance(accountNo); getMiniStatement(accountNo)	Perform customer transactions.
Persistence	loadAccounts(); saveAccount(account); saveTransaction(txn);	Read/write local files.

Function names are illustrative and may be changed during implementation; the important design constraint is the separation of responsibilities.

4.3 Sequence Diagram — Employee Creates Account

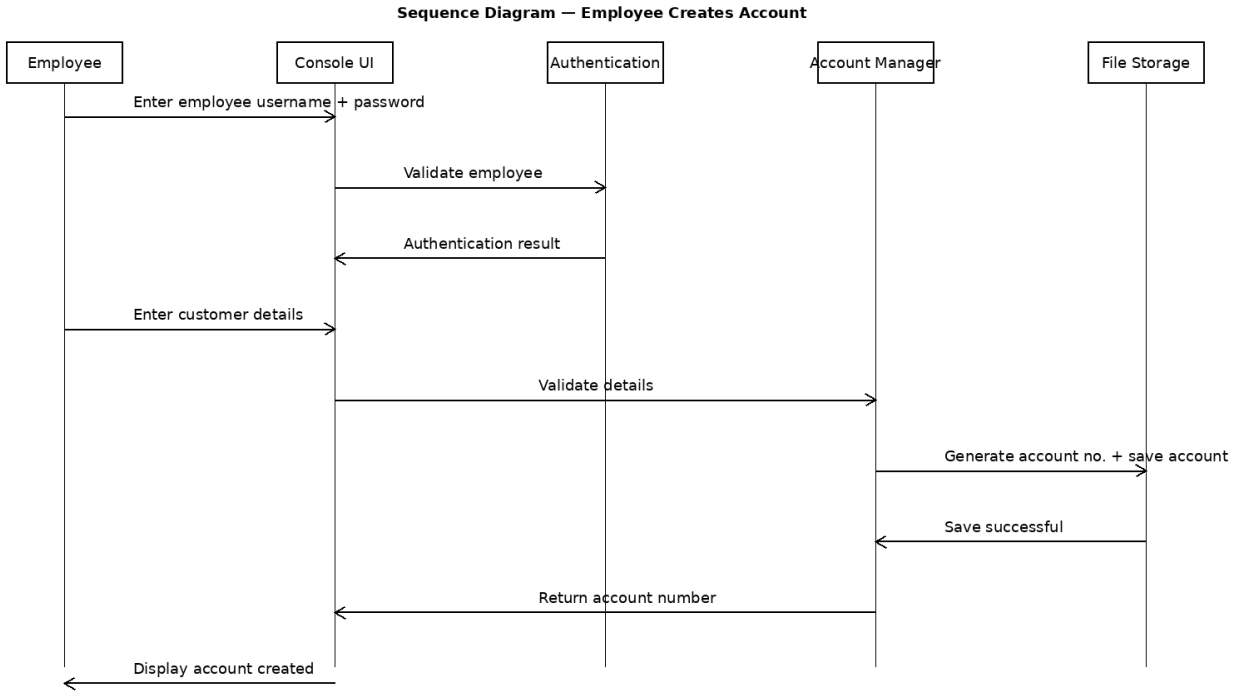


Figure 2. Account creation sequence.

4.4 Sequence Diagram — Customer Deposit

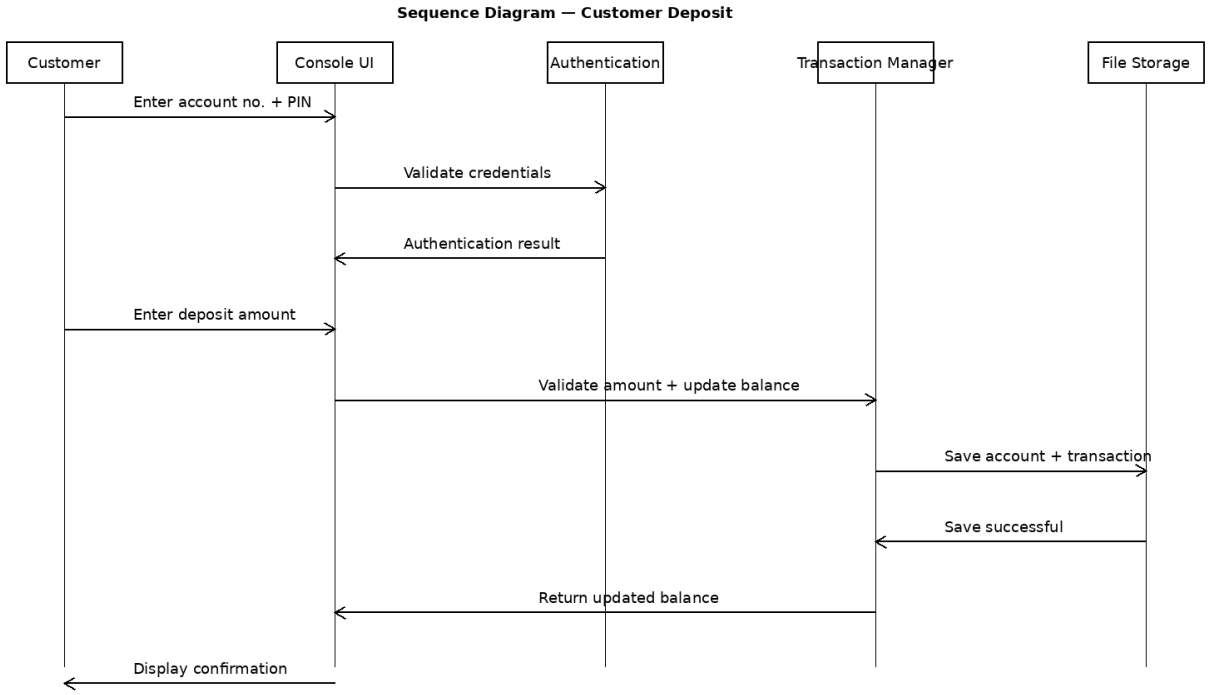


Figure 3. Deposit sequence.

5. Error Handling

Error	Handling
Invalid login	Display authentication error; return to login/menu.
Invalid account number	Display account-not-found message; do not modify data.
Duplicate account data	Reject creation and display an appropriate message.
Invalid/negative/zero amount	Reject operation; balance remains unchanged.
Insufficient balance	Reject withdrawal; display insufficient-balance message.
File read/write failure	Display storage error and do not report the operation as successful.
Invalid menu choice	Display a simple error and request another choice.

6. Design Constraints and Deliberate Scope

To keep the implementation achievable, the design does not include a database server, REST API, GUI framework, network communication, encryption infrastructure, concurrency control, real payment processing or production-grade banking compliance.

7. Design-to-Requirement Traceability

Requirement Group	Design Element	Verification
FR-1, FR-2	Account Management + File	TC-01

	Storage	
FR-3, FR-4, FR-5, FR-6	Transaction Management + Transaction Log	TC-03 to TC-08
FR-7, FR-8, FR-9	Account Management	TC-09 to TC-11
FR-10, FR-11, SEC-1 to SEC-3	Authentication	TC-12, TC-13
FR-12, SEC-4	Validation + Error Handling	TC-04, TC-06, TC-14
FR-13, NFR-6	File Storage	TC-15
NFR-1, NFR-2	Console UI + local modules	TC-16, TC-17
NFR-4, NFR-5	Modular code + compiler	TC-18

8. Implementation Plan

7. Create Account and Transaction structures/classes.
8. Implement file load/save functions.
9. Implement employee and customer authentication.
10. Implement employee account operations.
11. Implement customer transaction operations.
12. Connect all modules through the console menus.
13. Run the test cases and fix only defects within the stated scope.